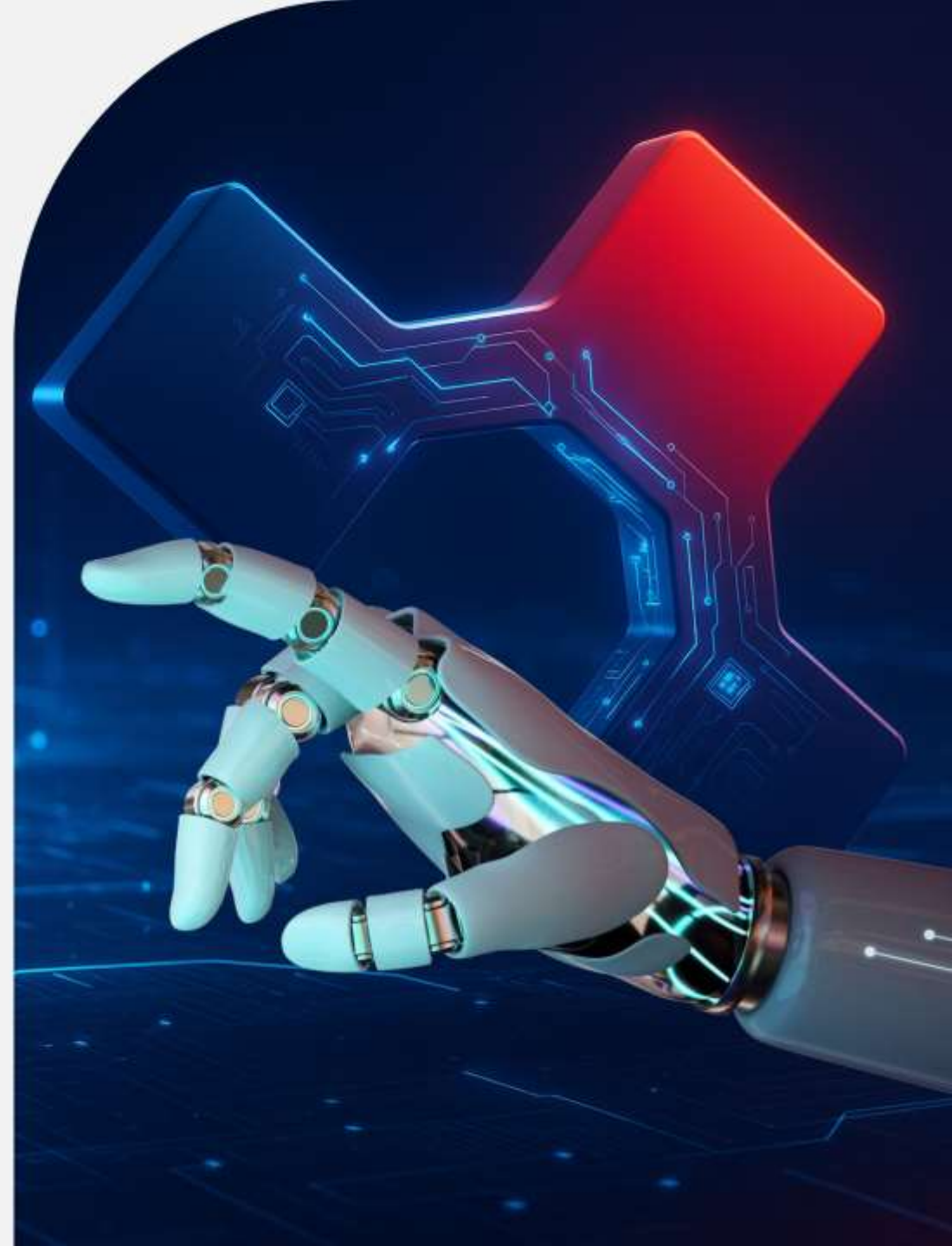


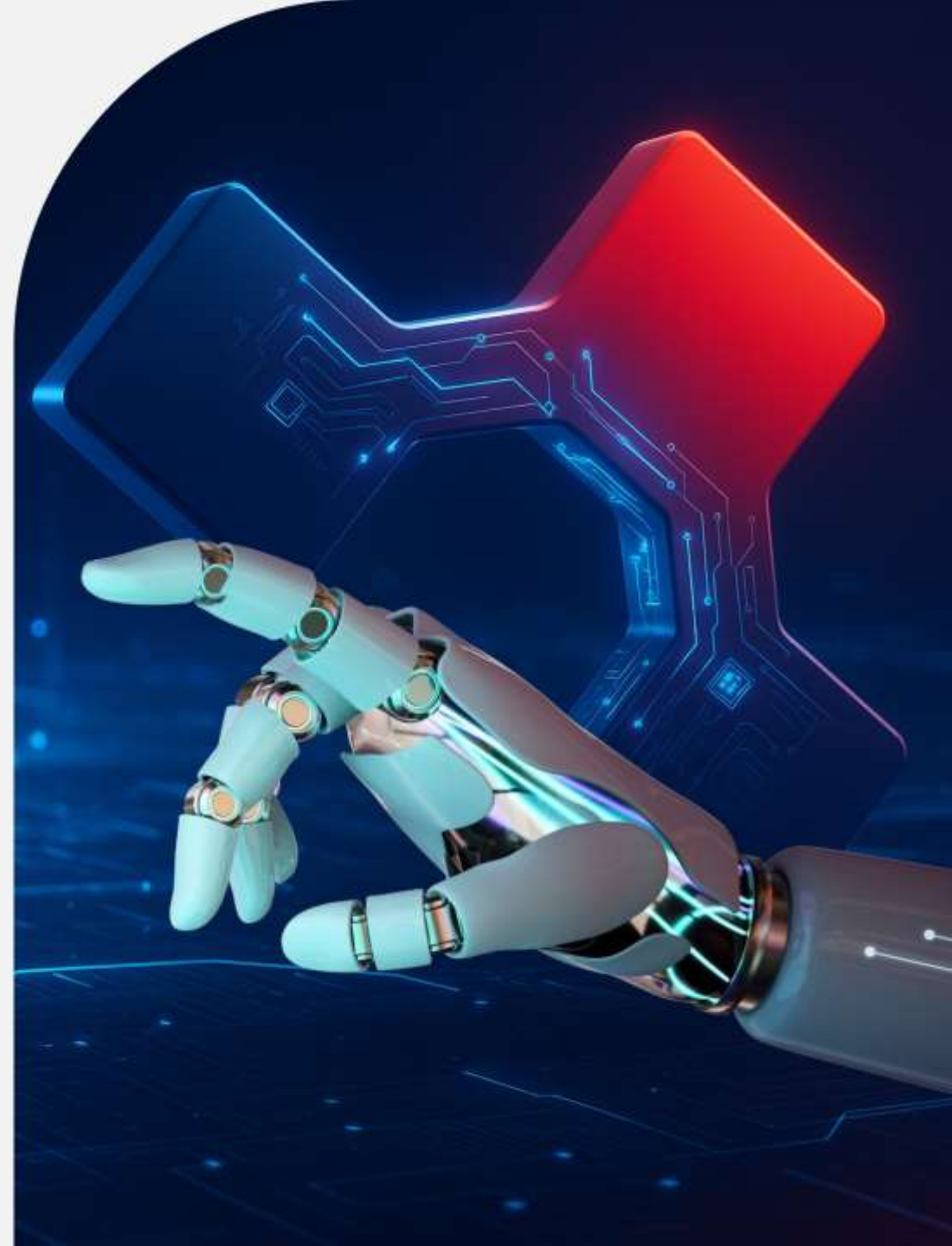


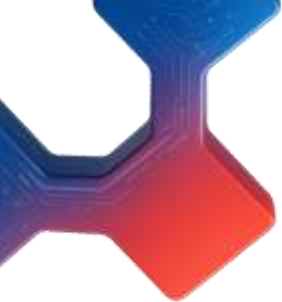
Núcleo de Capacitação em Inteligência Artificial



Treinando Modelos

**Regressão Linear; Equação Normal; Complexidade Computacional;
Gradiente Descendente; Gradiente Descendente em Batch (em lote);
Gradiente Descendente Estocástico.**

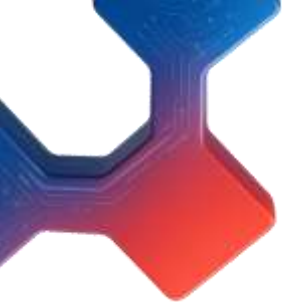




1. Introdução

1. Trabalhamos no problema de previsão médio de casas, e usamos os seguintes modelos:
 - **LinearRegression**
 - **DecisionTreeRegressor**
 - **RandomForestRegressor**
 - **Support Vector Regression.**
2. Trabalhamos no problema classificação de imagens de dígitos , e usamos os seguintes modelos:
 - **SGDClassifier**
 - **KNeighborsClassifier**
 - **RandomForestRegressor**
 - **Support Vector Classification .**





1. Introdução

Abordaremos duas maneiras diferentes de treinar uma **regressão linear**:

- Usando uma equação direta de forma fechada que calcula os parâmetros do modelo que melhor se ajustam ao modelo no conjunto de treinamento (ou seja, os parâmetros do modelo que minimizam a função de custo no conjunto de treinamento).
- Usando uma abordagem de otimização, chamada **gradiente descendente**, os parâmetros do modelo são ajustados gradualmente para minimizar a função de custo no conjunto de treinamento, eventualmente convergindo para o mesmo conjunto de parâmetros obtido pelo método de solução analítica.

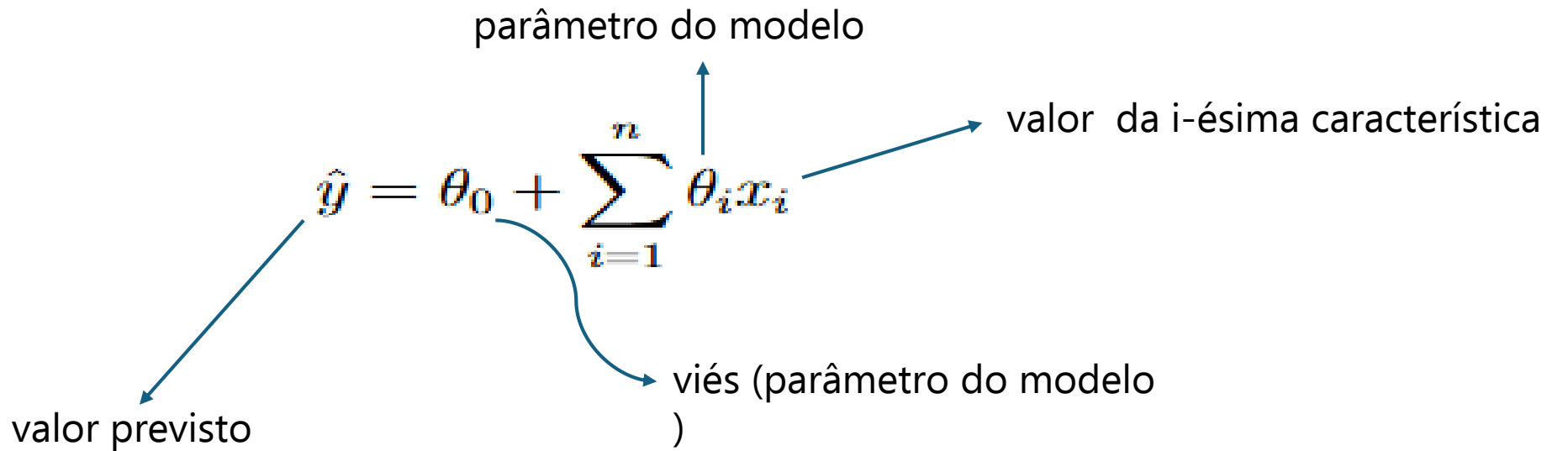
1. Gradiente descendente em batch (em lote)

2. Gradiente descendente estocástico

3. Gradiente descendente em mini-batch.



2. Regressão Linear



The diagram shows the linear regression equation $\hat{y} = \theta_0 + \sum_{i=1}^n \theta_i x_i$ with several annotations:

- An arrow points from $\hat{y}$ to the text "valor previsto".
- An arrow points from θ_0 to the text "viés (parâmetro do modelo)".
- An arrow points from θ_i to the text "parâmetro do modelo".
- An arrow points from x_i to the text "valor da i-ésima característica".

Isso pode ser escrito de uma forma vetorizada como vamos mostrar a seguir.



2. Regressão Linear

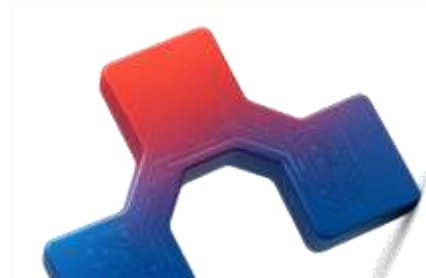
$$\hat{y} = h_{\theta}(\mathbf{x}) = [\theta_0 \quad \theta_1 \quad \theta_2 \quad \cdots \quad \theta_n] \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Esse é o modelo de regressão linear da forma vetorizada. Mas como vamos treiná-lo ? Basta encontrar os parâmetros que minimize a seguinte função de custo.

$$MSE(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

Diagram illustrating the components of the Mean Squared Error (MSE) function:

- $\mathbf{X}$: conjunto de treinamento
- θ : vetor dos parâmetros
- m : número de amostras
- $\mathbf{x}^{(i)}$: entradas
- $y^{(i)}$: rótulo



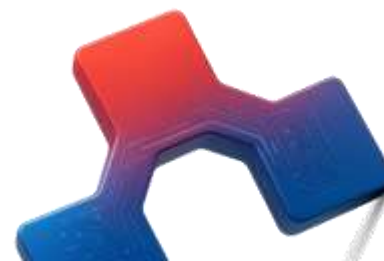
2.2 Equação Normal

Vamos encontrar os valores que minimiza a função de custo MSE de maneira geral. Para isso vamos formular o problema de forma matricial.

$$\begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \theta_0 + \theta_1 x_{11} + \cdots + \theta_n x_{1n} \\ \theta_0 + \theta_1 x_{21} + \cdots + \theta_n x_{2n} \\ \vdots \\ \theta_0 + \theta_1 x_{m1} + \cdots + \theta_n x_{mn} \end{bmatrix}$$

A função de custo é definida da seguinte forma:

$$\text{MSE}(\theta) = \frac{1}{m} (X\theta - y)^\top (X\theta - y)$$



2.2 Equação Normal

Para determinar os parâmetros ótimos precisamos fazer algumas contínuas com derivadas de matrizes:

$$\text{MSE}(\theta) = \frac{1}{m} (X\theta - y)^T (X\theta - y)$$

$$\begin{aligned} (X\theta - y)^T (X\theta - y) &= (X\theta)^T X\theta - (X\theta)^T y - y^T X\theta + y^T y \\ &\quad - (X\theta)^T y - y^T X\theta = -2y^T X\theta \end{aligned}$$

$$\text{MSE}(\theta) = \frac{1}{m} (\theta^T X^T X\theta - 2y^T X\theta + y^T y)$$

$$\frac{\partial \text{MSE}(\theta)}{\partial \theta} = \frac{1}{m} (2X^T X\theta - 2X^T y) = \frac{2}{m} (X^T X\theta - X^T y)$$

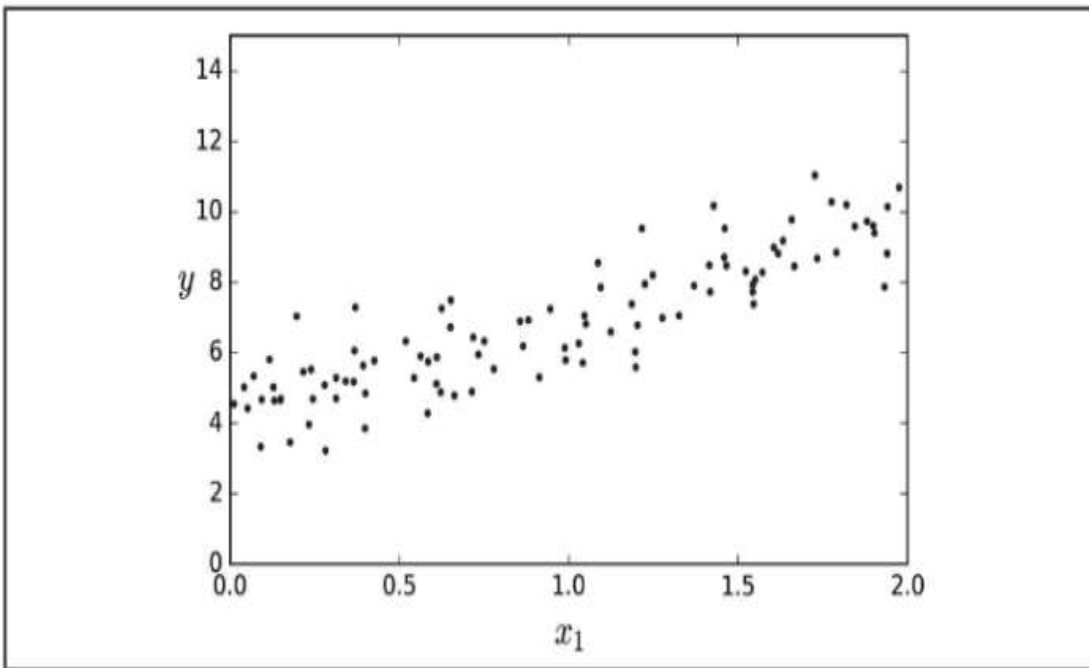
$$X^T X\theta - X^T y = 0$$

$$X^T X\theta = X^T y$$

$$\theta = (X^T X)^{-1} X^T y$$

2.2 Equação Normal

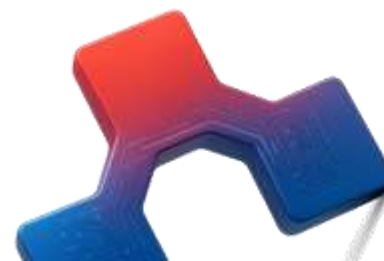
Achamos os valores que minimizam a função custo MSE para regressão, ela conhecida na literatura por **equação normal**. Agora vamos gerar alguns dados lineares para testar essa equação.



```
import numpy as np
```

```
X = 2 * np.random.rand(100, 1)
```

```
y = 4 + 3 * X + np.random.randn(100, 1)
```



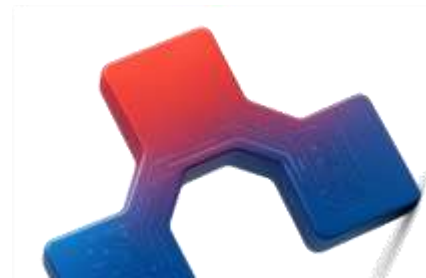
2.2 Equação Normal

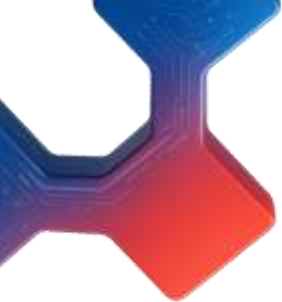
Agora vamos achar os parâmetros que definem a solução do problema.

```
X_b = np.c_[np.ones((100, 1)), X] # adiciona x0 = 1 a cada instância  
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

A função vigente que usamos para gerar os dados é $y = 4 + 3x_1 + \text{ruído gaussiano}$. Veja se encontrou :

```
>>> theta_best  
array([[ 4.21509616],  
       [ 2.77011339]])
```





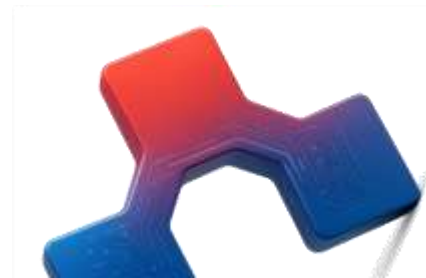
2.2 Equação Normal

Chegamos um pouco perto. O ruído tornou impossível recuperar os parâmetros exatos da função original. Agora podemos fazer algumas previsões usando os parâmetros encontrados.

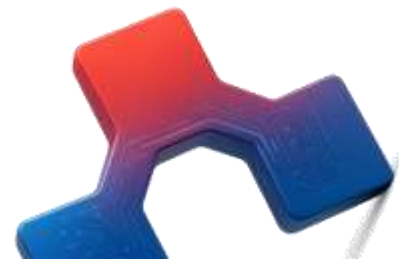
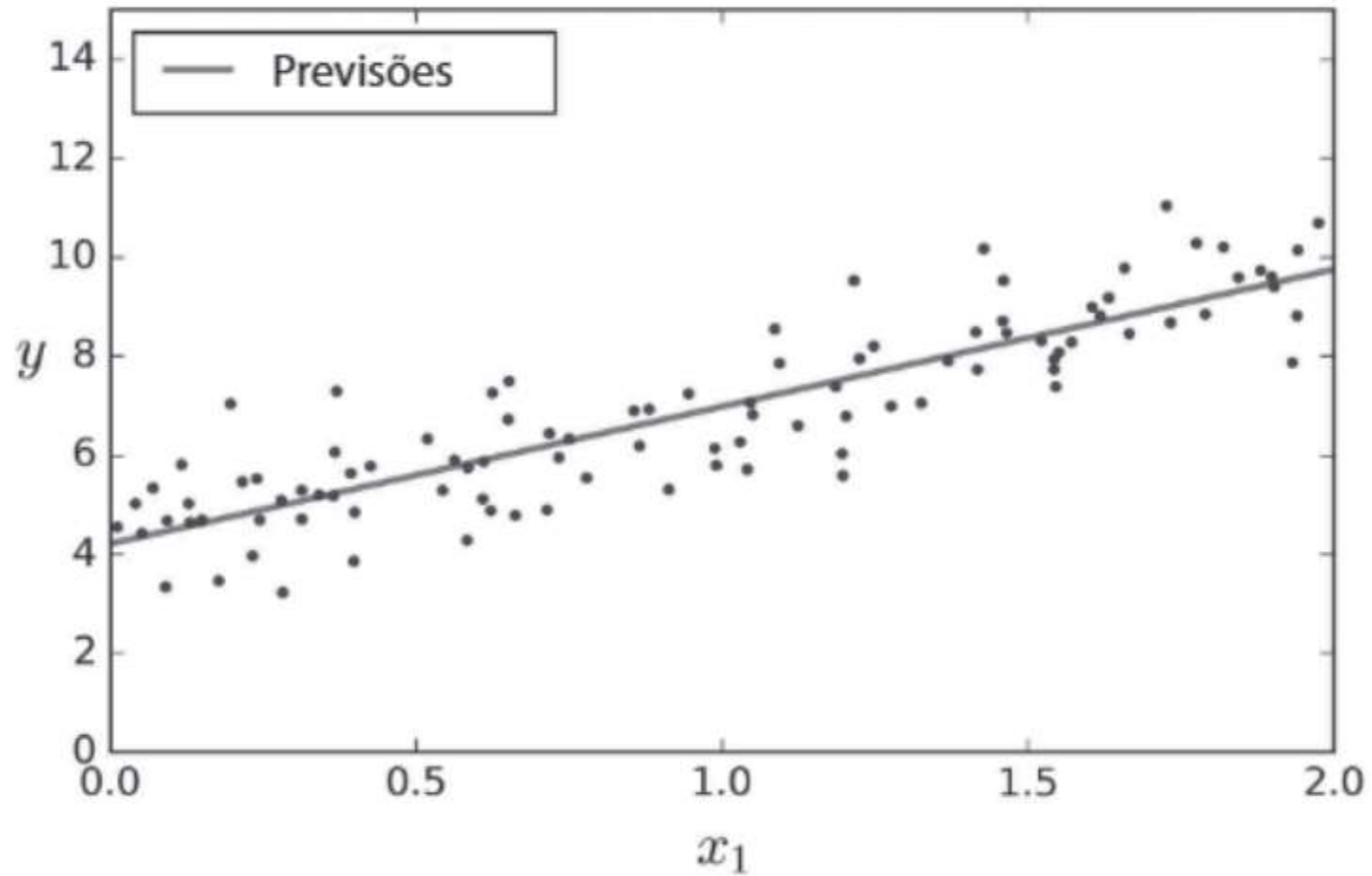
```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # adiciona x0 = 1 a cada instância
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[ 4.21509616],
       [ 9.75532293]])
```

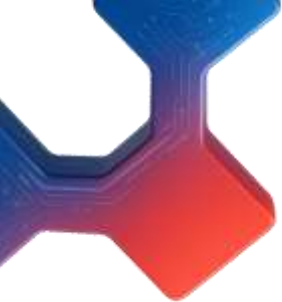
Vamos plotar as previsões:

```
plt.plot(X_new, y_predict, "r-")
plt.plot(X, y, "b.")
plt.axis([0, 2, 0, 15])
plt.show()
```



2.2 Equação Normal

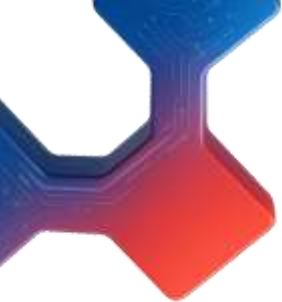




2.3 Scikit- Learn

O código equivalente utilizando o Scikit-learn se parece com isso:

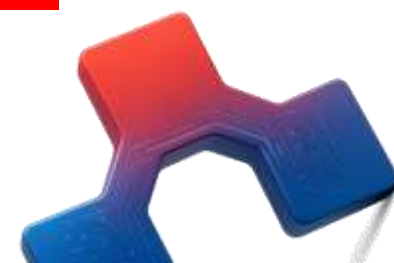
```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([ 4.21509616]), array([[ 2.77011339]]))
>>> lin_reg.predict(X_new)
array([[ 4.21509616],
       [ 9.75532293]])
```



2.3 Complexidade Computacional

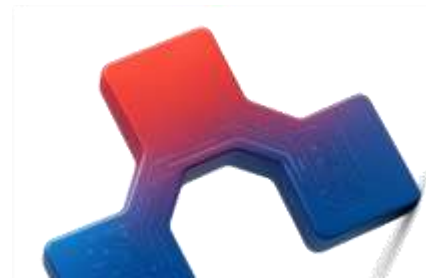
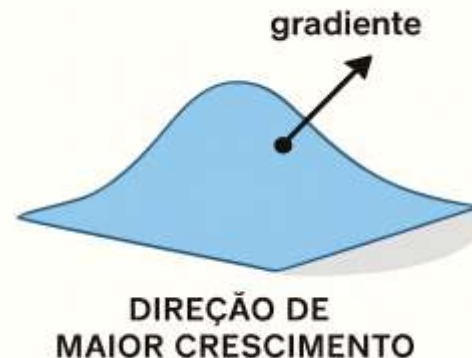
- É possível calcular os parâmetros que definem a solução de um problema de regressão linear usando a solução das equações normais.
- Quando o número de características aumenta, cresce também a dimensão da matriz que precisa ser invertida. Como o cálculo da inversa de uma matriz grande exige muitas operações, o método pode se tornar **computacionalmente caro e numericamente instável**.
- Encarando pelo lado positivo, trabalhamos com uma equação linear em relação ao número de instâncias no conjunto de treinamento, de modo que lida eficientemente com grandes conjuntos de treinamento, desde que possam caber na memória.

Diante da dificuldade de inverter matrizes grandes, surge a seguinte questão: **existe uma maneira de encontrar os parâmetros da regressão linear sem precisar calcular a inversa diretamente?**



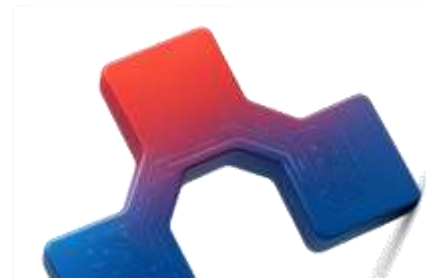
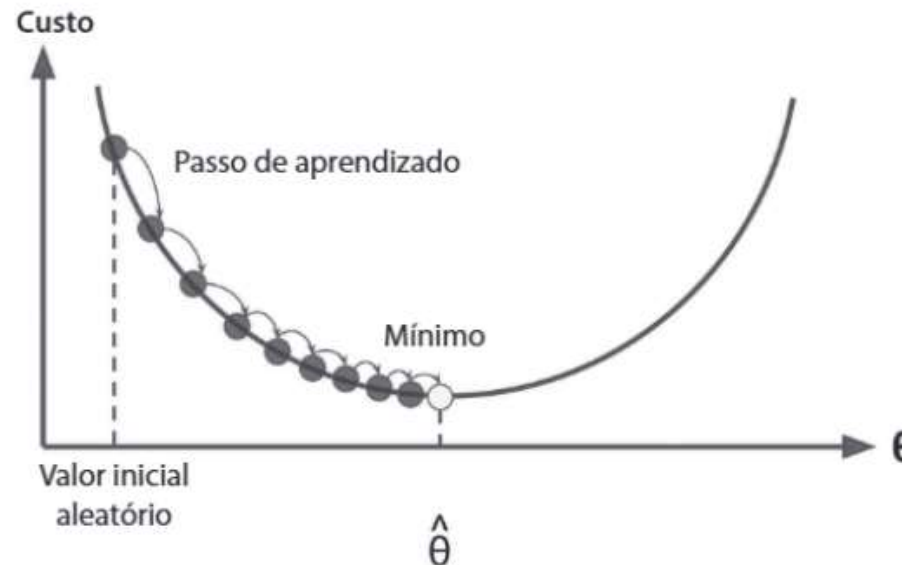
3. Gradiente Descendente

- O **gradiente descendente** é um algoritmo de otimização muito genérico capaz de encontrar ótimas soluções para uma ampla gama de problemas. A ideia desse algoritmo é ajustar iterativamente os parâmetros para minimizar uma função de custo.
- O **gradiente** é um vetor que indica a direção na qual uma função aumenta mais rápido. Ele mostra **qual caminho seguir para que a função cresça mais rapidamente** e o tamanho do vetor indica **o quanto a função cresce nessa direção**.



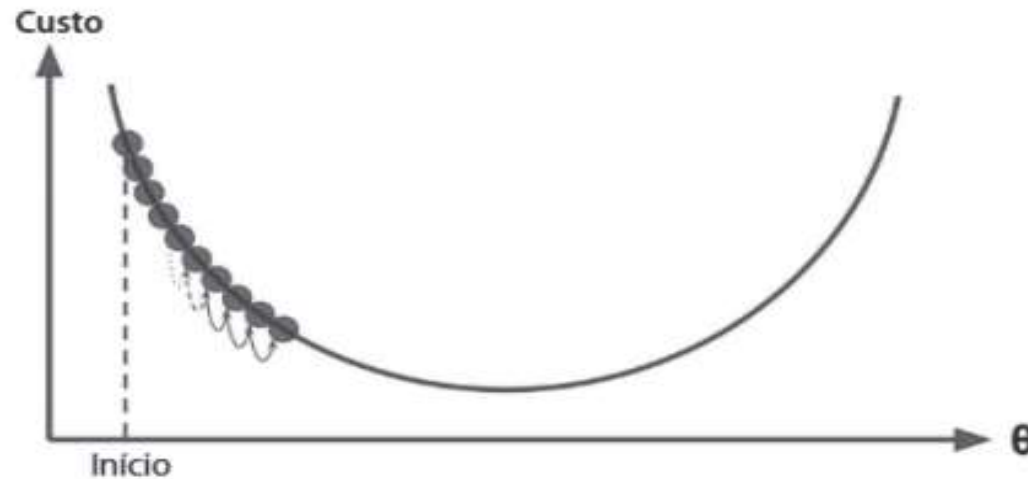
3. Gradiente Descendente

- Concretamente, você começa com valores aleatórios (isto é chamado de inicialização aleatória), e então melhora o gradiente, dando um pequeno passo por vez, cada passo tentando diminuir uma função custo (por exemplo, a MSE), até que o algoritmo convirja para um mínimo.



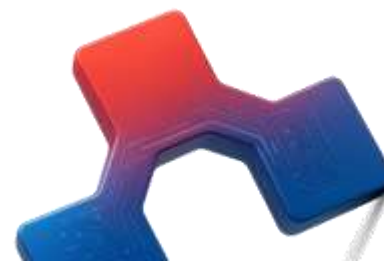
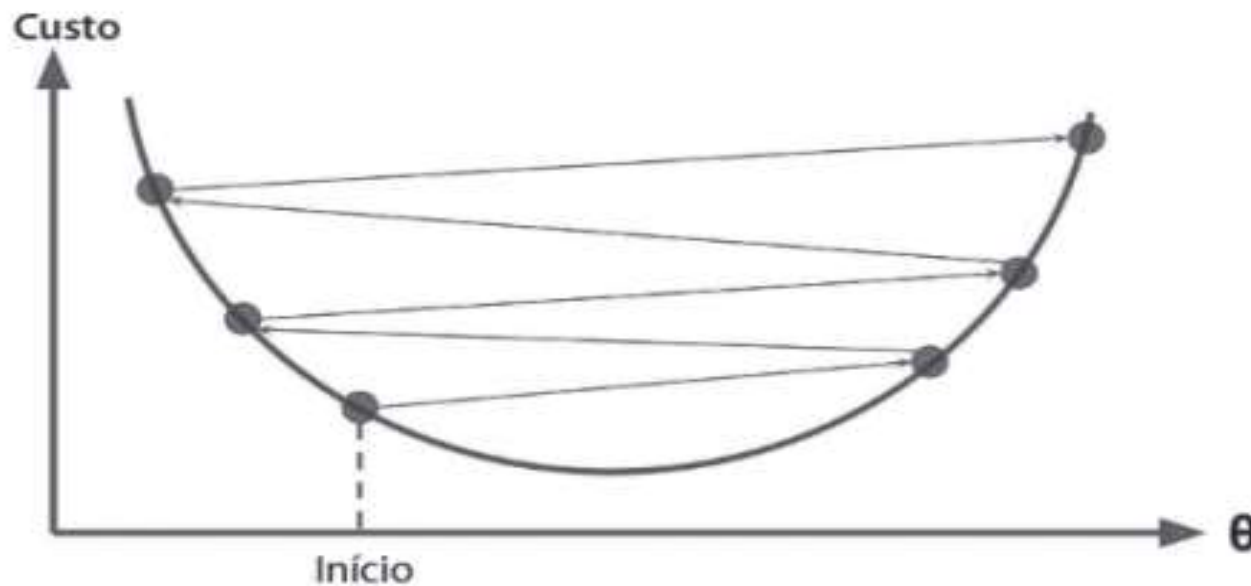
3. Gradiente Descendente

- O tamanho dos passos é um parâmetro importante do gradiente descendente, determinado pelo hiperparâmetro **taxa de aprendizado**.
- Se a taxa de aprendizado for muito pequena, o algoritmo terá que passar por muitas iterações para convergir, o que demora muito.



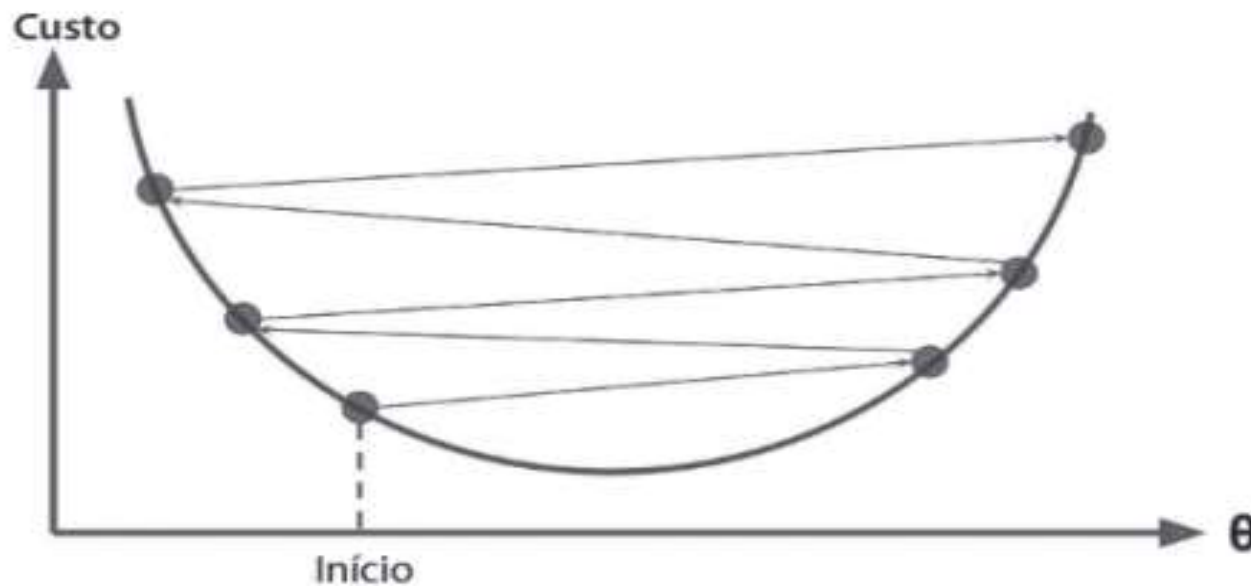
3. Gradiente Descendente

- Por outro lado, se a taxa de aprendizado for muito alta, você pode atravessar o vale e acabar do outro lado, possivelmente até mais alto do que estava antes. Isso pode tornar o algoritmo divergente com valores cada vez maiores, não encontrando uma boa solução.



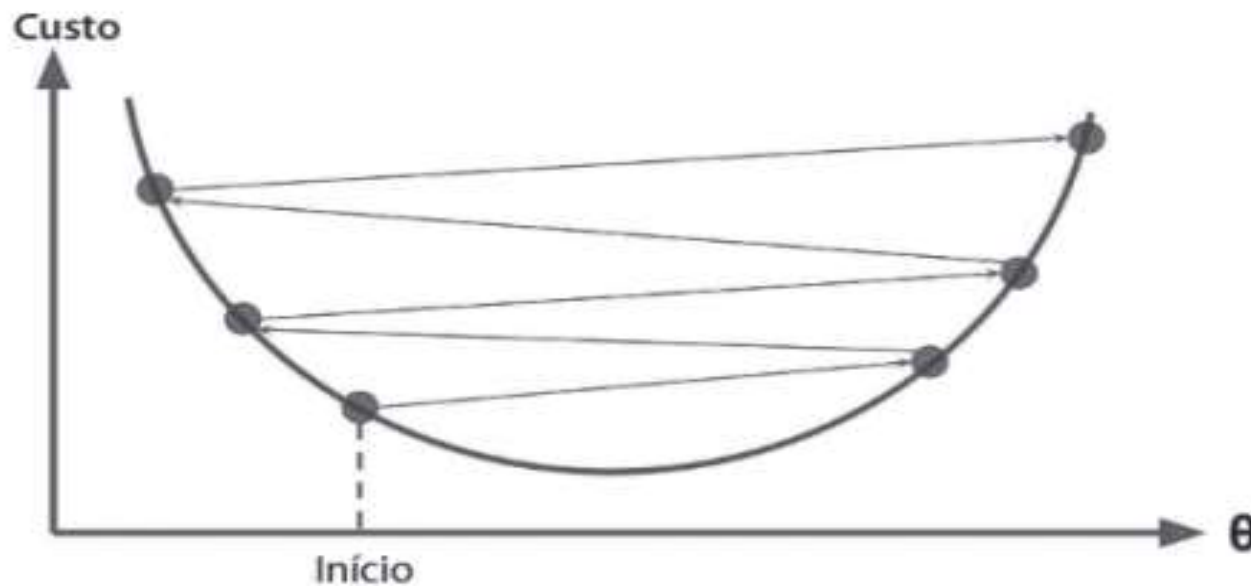
3. Gradiente Descendente

- Por outro lado, se a taxa de aprendizado for muito alta, você pode atravessar o vale e acabar do outro lado, possivelmente até mais alto do que estava antes. Isso pode tornar o algoritmo divergente com valores cada vez maiores, não encontrando uma boa solução.



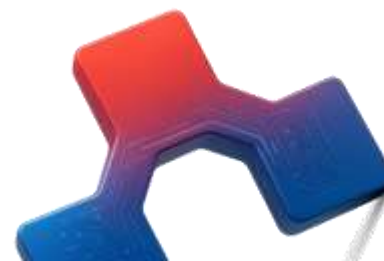
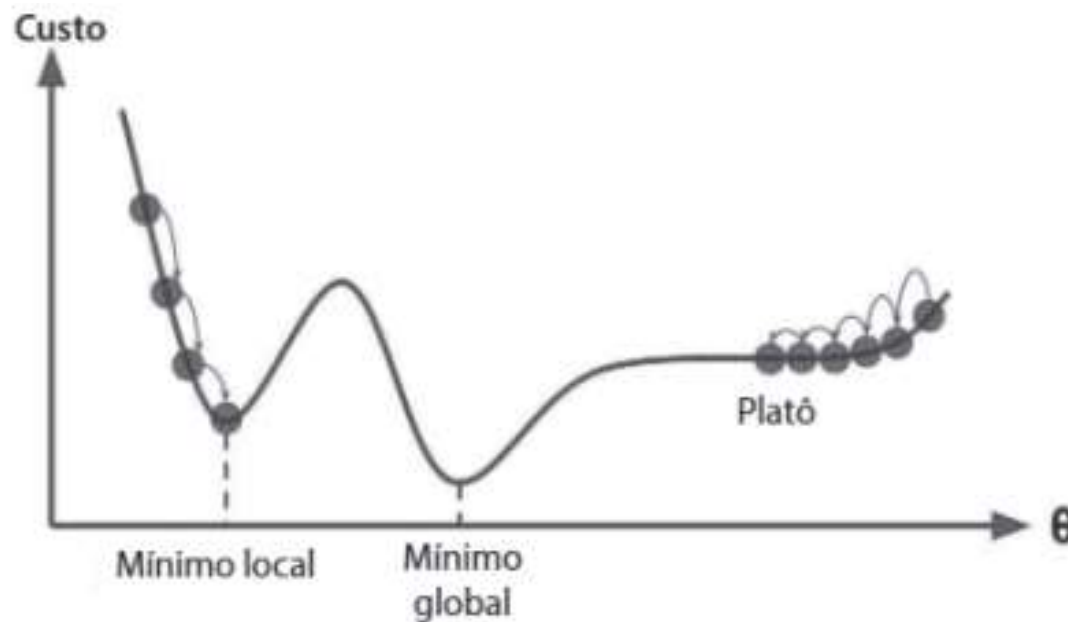
3. Gradiente Descendente

- Por outro lado, se a taxa de aprendizado for muito alta, você pode atravessar o vale e acabar do outro lado, possivelmente até mais alto do que estava antes. Isso pode tornar o algoritmo divergente com valores cada vez maiores, não encontrando uma boa solução.



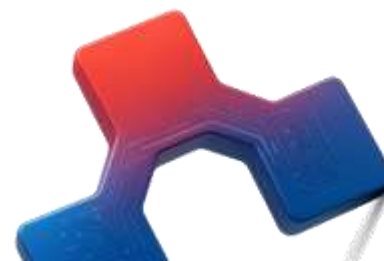
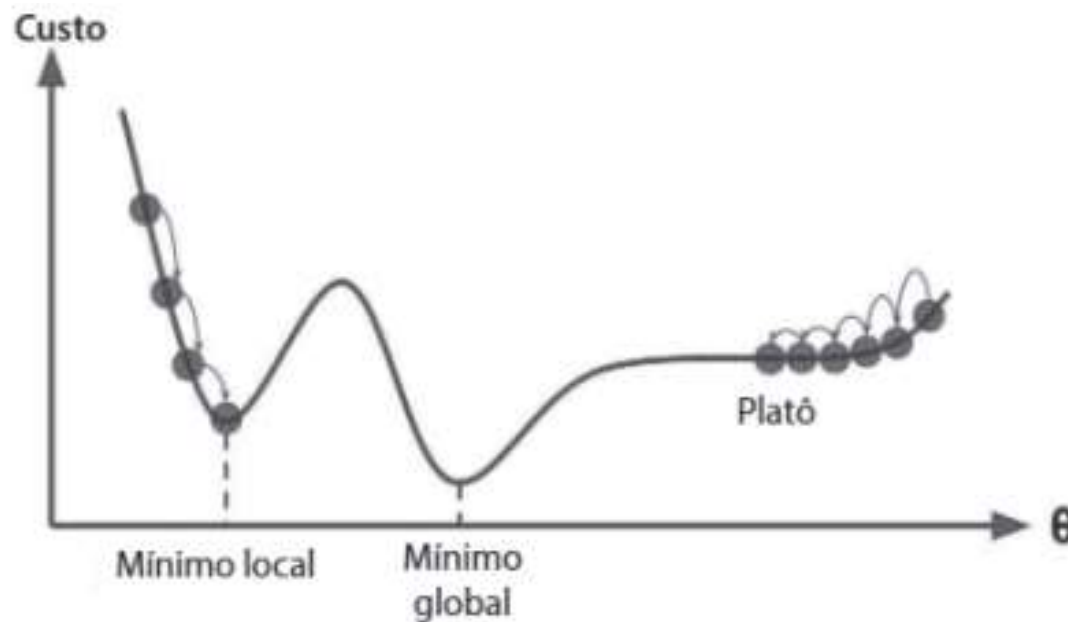
3. Gradiente Descendente

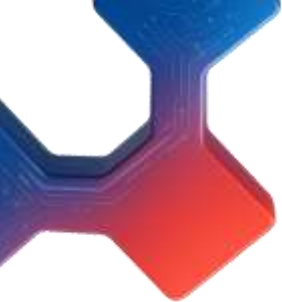
- Finalmente, nem todas as funções de custo se parecem com “tigelas” regulares. Podem haver buracos, cumes, planaltos e todo tipo irregular de terreno fazendo com que a convergência ao mínimo seja muito difícil.



3. Gradiente Descendente

- Finalmente, nem todas as funções de custo se parecem com “tigelas” regulares. Podem haver buracos, cumes, planaltos e todo tipo irregular de terreno fazendo com que a convergência ao mínimo seja muito difícil.



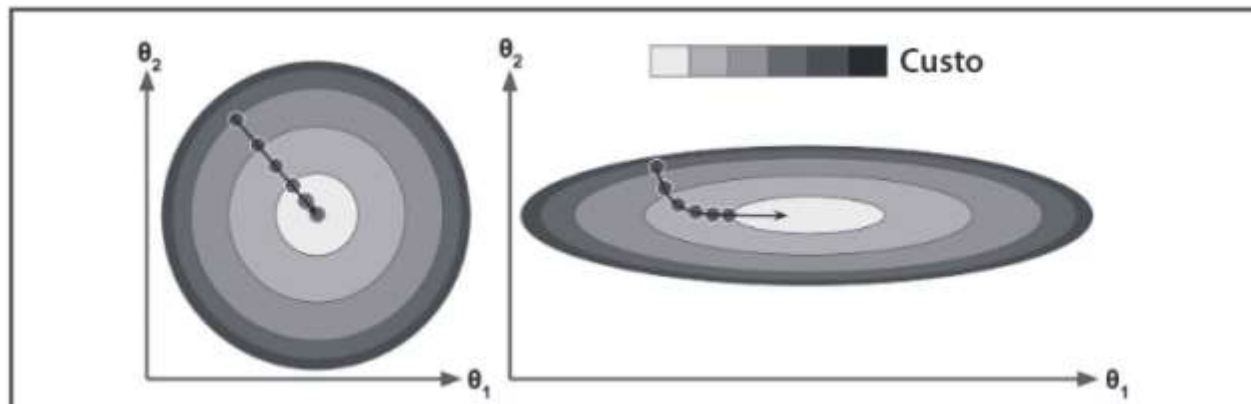


3. Gradiente Descendente

- A figura acima mostra os dois desafios do gradiente descendente.
 1. Se a inicialização for aleatória iniciar o algoritmo à esquerda, ela converge para um mínimo local, o que não é tão bom quanto o mínimo global.
 2. Se começar pela direita, então demorará muito para atravessar o planalto e, se você parar cedo demais, nunca alcançará o mínimo global.
- Felizmente, a função de custo MSE para um modelo de regressão linear é uma **função convexa**. Isso implica que não há mínimos locais, apenas um mínimo global. É também uma função contínua com uma inclinação que nunca se altera abruptamente. Estes dois fatos geram uma ótima consequência: O algoritmo tem a garantia de se aproximar arbitrariamente do mínimo global (**se você esperar o tempo suficiente e se a taxa de aprendizado não for muito alta**).

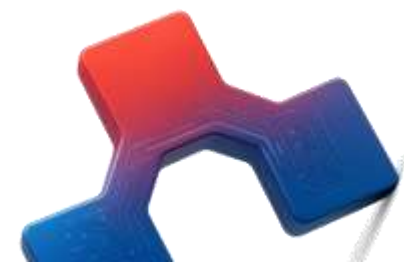
3. Gradiente Descendente

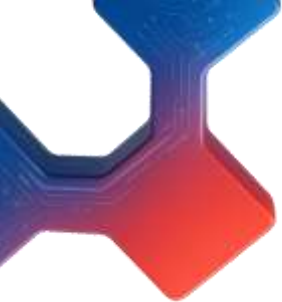
- A figura acima mostra os dois desafios do gradiente descendente.
 1. Se a inicialização for aleatória iniciar o algoritmo à esquerda, ela converge para um mínimo local, o que não é tão bom quanto o mínimo global.
 2. Se começar pela direita, então demorará muito para atravessar o planalto e, se você parar cedo demais, nunca alcançará o mínimo global.
- Gradiente descendente com e sem escalonamento das características.



3. Gradiente Descendente

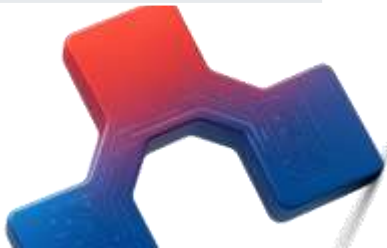
- Através da figura acima pode-se concluir que treinar um modelo significa procurar uma combinação de parâmetros do modelo que minimizem uma função de custo (em relação ao conjunto de treinamento). É uma pesquisa no espaço de parâmetro do modelo: quanto mais parâmetros um modelo possui, mais dimensões este espaço possui, e mais difícil será a busca:





3.1 Método direto vs. método iterativo para ajuste de parâmetros

Aspecto	Solução Analítica (fechada)	Gradiente Descendente (iterativo)
Cálculo dos parâmetros	Fórmula direta, resultado imediato	Ajuste gradual, passo a passo
Tempo de execução	Rápido para datasets pequenos/médios	Depende do número de iterações e tamanho do dataset
Complexidade do modelo	Funciona bem para modelos simples (ex.: regressão linear)	Funciona para modelos simples e complexos (ex.: redes neurais)
Memória / dados	Precisa processar todos os dados de uma vez	Pode processar dados em lotes ou mini-lotes, mais eficiente para grandes datasets
Resultado final	Parâmetros ótimos diretamente	Parâmetros ótimos alcançados gradualmente (convergência)
Função de custo	Deve permitir solução fechada (ex.: convexa)	Pode lidar com funções de custo complexas e não-lineares



4. Gradiente Descendente em Batch (em lote)

- O gradiente descendente em lote é um método de otimização usado em aprendizado de máquina , que calcula o gradiente da função custo **em todo o conjunto de dados** em cada etapa. Esse procedimento garante uma atualização estável dos parâmetros do modelo em direção à minimização do erro.

Com funciona ?

1. Calcula-se a função custo
2. Calcula-se o gradiente em relação aos parâmetros do modelo, usando todos os exemplos
3. Atualiza-se os parâmetros:

$$\theta := \theta - \eta \nabla \text{MSE}(\theta)$$

Tamanho do
passo em cada
iteração

repetindo o processo de forma iterativa até a convergência.



4. Gradiente Descendente em Batch (em lote)

- Implementação rápida desse algoritmo

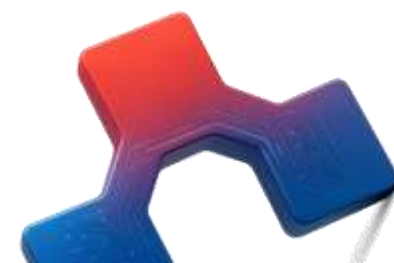
```
eta = 0.1 # taxa de aprendizado
n_iterations = 1000
m = 100

theta = np.random.randn(2,1) # inicialização aleatória

for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients
```

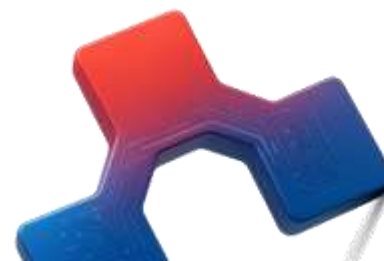
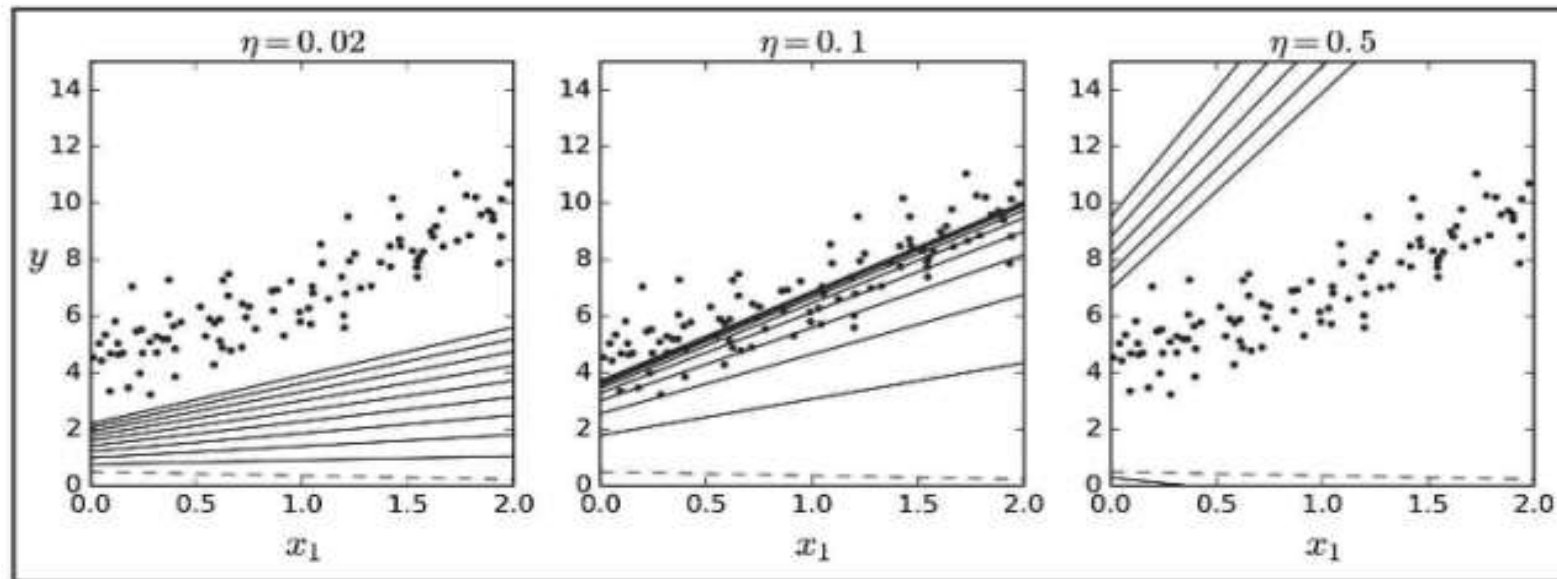
- Observe o parâmetro ótimo resultante

```
>>> theta
array([[ 4.21509616],
       [ 2.77011339]])
```

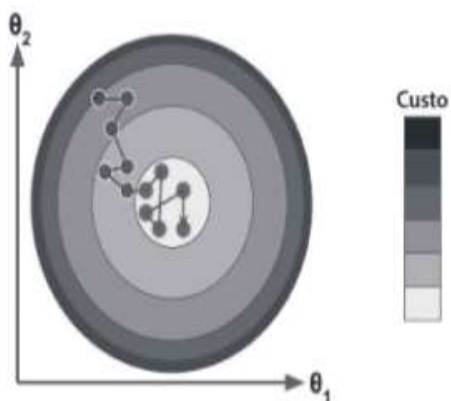


4. Gradiente Descendente em Batch (em lote)

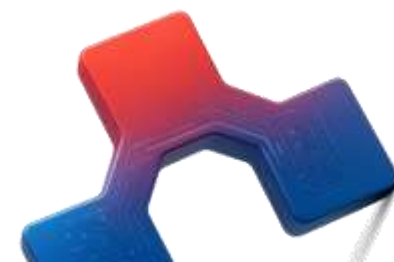
- Vemos que o algoritmo funcionou perfeitamente. Mas a pergunta que não quer calar: E se usar uma taxa de aprendizado diferente ? Faça você esse teste !!!!!



5. Gradiente Descendente Estocástico



- O principal problema do gradiente descendente em lote é o fato de que ele utiliza todo o conjunto de treinamento para calcular os gradientes em cada passo, o que o torna muito lento quando o conjunto for grande
- No extremo oposto, o **gradiente descendente estocástico** escolhe uma instância aleatória no conjunto de treinamento em cada etapa e calcula os gradientes baseado apenas nesta instância.
- Quando a função custo é muito irregular, isso pode, na verdade, ajudar o algoritmo a pular fora do mínimo local, de modo que o gradiente descendente estocástico terá uma chance maior de encontrar o mínimo global do que o gradiente descendente em lote.



5. Gradiente Descendente Estocástico

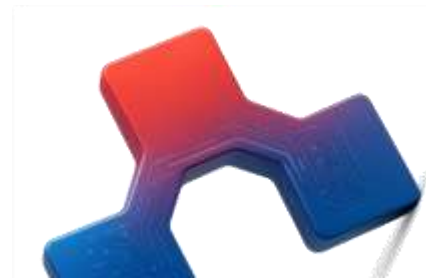
- Utilizando um simples cronograma de aprendizado, este código implementa o gradiente descendente estocástico:

```
n_epochs = 50
t0, t1 = 5, 50 # hiperparâmetros de aprendizado

def learning_schedule(t):
    return t0 / (t + t1)

theta = np.random.randn(2,1) # inicialização aleatória

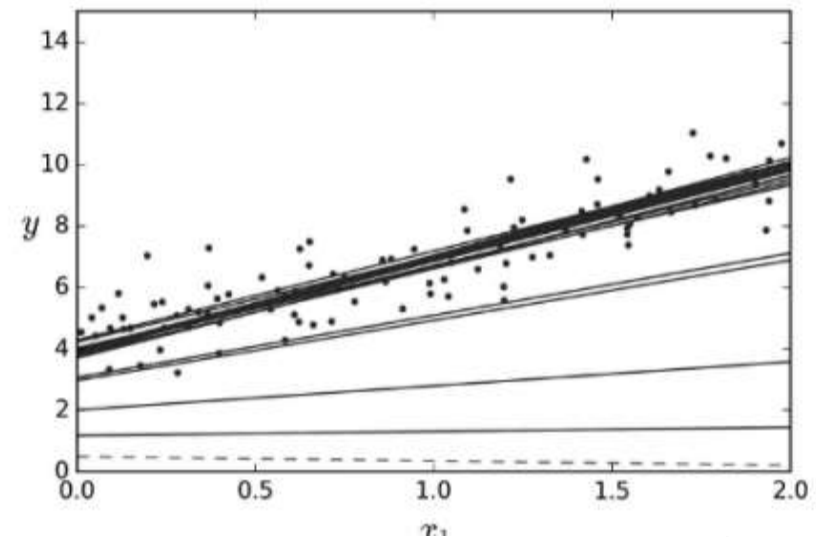
for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients
```

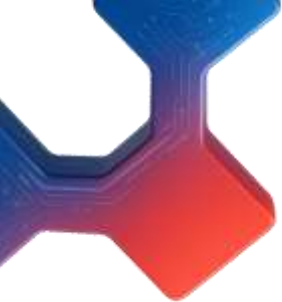


5. Gradiente Descendente Estocástico

- Por convenção, replicamos por m iterações; cada rodada é chamada época. Enquanto o código gradiente descendente em lote replica mil vezes pelo conjunto de treinamento, este código passa apenas 50 vezes pelo conjunto e atinge uma boa solução:

```
>>> theta  
array([[ 4.21076011],  
       [ 2.74856079]])
```





5.2 Regressão Linear usando SGD com Scikit- Learn

- Para executar a regressão linear usando SGD com scikit-learn, podemos aplicar a classe SGDRegressor, que otimiza o padrão da função custo de erro quadrático.

```
from sklearn.linear_model import SGDRegressor  
sgd_reg = SGDRegressor(n_iter=50, penalty=None, eta0=0.1)  
sgd_reg.fit(X, y.ravel())
```

- Mais uma vez podemos ver que encontramos uma solução parecida da retornada pelo método analítico do gradiente descendente.

